

Lecture 10 - DSA

6 May 2026

Sorted Containers

Priority Queue

Stack
Queue } order matters

$$h(k) \rightarrow p$$

$$h(k_2) \rightarrow p$$

$$h(k) = k \% m$$

(definition of hash fct.)

Dynamic Array Iterator + LinkedList Iterator.

Iterator HT:

ht: Hash Table

currentPos: Integer

currentNode: $\uparrow$ Node

$\rightarrow$ no longer valid when currentPos is

Coalesced chaining:

$m=13$.

	0	1	2	3	4	5	6	7	8	9	10	11	12
elems	18	13	15	16	31	5	26						
next	1	4	-1	-1	+6	0	-1	-1	-1	-1	-1	-1	-1

(*)

firstEmpty = $\emptyset$ 1 4 6 7 we're making a linked list.

$\uparrow$
13 on first found

$O(m)$

Key	26
Hash	0

$\rightarrow$ starts on 0 : index 0 : elems: 18, next: 1

index 1 : elems: 13, next: 4

index 4 : elems: 31, next: +6

index 6 : elems: 26 ; next: -1

HashTable:

T: TKey[]

next: Integer[]

m: Integer

firstEmpty: Integer

h: TFunction

! You need to add each elem again in the hash function, not just copy like in DA.

!

Search for 31:

$h(31) = 31 \% 13 = 5 \Rightarrow$ go to position 5 $\rightarrow$ position 0 $\rightarrow$ pos 1 $\rightarrow$ pos 4
 $\Rightarrow$ we found it.

$h(39) = 39 \% 13 = 0 \rightarrow$ pos 0 $\rightarrow$ pos 1 $\rightarrow$ pos 4 $\rightarrow$ pos 6 $\rightarrow$ pos -1
 $\Rightarrow$ we did not find it.

Start with hash function, follow the links!

ex1:

0	1	2	3	4
3	11		8	
-1	-1	-1	0	-1

11 8 3

$11 \% 5 \Rightarrow$ pos 1

$8 \% 5 \Rightarrow$ pos 3

$3 \% 5 \Rightarrow$ pos 3 $\Rightarrow$ occupied $\Rightarrow$ point to 0 $\Rightarrow$ put at pos 0.

ex2:

0	1	2	3	4
X	56	12	8	
-1	0	-1	-1	-1

$\Rightarrow$ move to -1.

$\Rightarrow$ remove 11 $\Rightarrow$

move

56 8 11 12

ex3:

0	1	2	3	4
20	11	56		
-1	-2	-1	-1	-1

$\Rightarrow$

...	56	...		
...	-1	...	...	...

11, 20, 56

ex4:

0	1	2	3	4
X	56	12	1	
X	3	-1	-1	-1

56, 11, 12, 1

ex5:

0	1	2	3	4
20	56		3	
-1	0	-1	-1	-1

Remove 18 from that table *

$$h(18) = 18 \% 13 = 5 \Rightarrow \text{pos. 5.}$$

! Some positions cannot be empty.

5 ~ 0 ~ 1 ~ 4 ~ 6 linked list.
↑
→

• search for the elem → found $\Rightarrow$ given pos ($\exists$ other elem that should be in this position?) $\Rightarrow$ you can safely remove this position

$\Rightarrow$ 18 13 15 16 31 5 26

$$h(5) = 5 \% 13 = 5$$

$\Rightarrow$ 18 should be on pos. 5

$\Rightarrow$ 18 13 15 16 31 18 26

$\Rightarrow$ elem that should be on pos 0? $\Rightarrow$ 13 should be on pos 0.

$\Rightarrow$ 13 18 15 16 31 18 26 ✓

$\Rightarrow$ safely remove pos 1. $\Rightarrow$ removal like on SL.

$\Rightarrow$ 13 15 16 31 18 26
-1 -1 -1 -1 0 -1

skip $\Rightarrow$ you need the previous.

Open Addressing

$$h(k, 0) \Rightarrow p_1$$

$$h(k, 1) \Rightarrow p_2$$

⋮

$$h(k, m-1) \Rightarrow p_m$$

↓
probe number

probe sequence (generates positions)

↳ contains half of permutations??

permutations: $m!$

1) I want a hash function that gives as many pos. as we can out of those permutations.